



Hanbi Jang, Client Programmer

PORTFOLIO

게임 플레이, 최적화, 네트워크 연결까지 게임의 재미를 직접 구현하는 게임 클라이언트 프로그래머

Name: 장한비 (Hanbi Jang)

Contact : 010-5227-5728 | hanbijang.dev@gmail.com

GitHub : github.com/HanbiJang

Core Strength : Unity 게임플레이 구현, 모바일 최적화, 멀티플레이 네트워크 연동 경험



About Me & Core Skills

기획부터 출시까지, 게임 개발 전 과정을 직접 경험

EA Korea 인턴십

멀티플레이 레이싱 게임 개발 및 대형 라이브 서비스 팀
과 협업 경험

상용화 경험

모바일 비주얼 노벨 <Dream Catcher> 구글 플레이
스토어 출시 및 서비스 유지보수

업무 효율화

사내 업무 자동화 스크립트 (Python, AHK) 설계로 운
영 효율 2배 향상



Engines

Unity



Unreal Engine 4



Languages

C#



C++



Java



Network

TCP Socket 통신

Photon PUN2

UE4 Replication 아키텍처

Projects Overview

01



Dream Catcher

커스텀 XML 파싱 엔진 직접 설계 및 플레이스토어 상용화

Unity C# XML

02



2D 소울라이크 전투 데모

FSM 아키텍처 기반의 정교한 2D 액션 RPG

Unity C# FSM

03



리듬 레이싱 멀티플레이 게임

C++ TCP 게임 서버 구축 및 Unity 클라이언트 연동

Unity C# TCP

04



물리 기반 멀티플레이 게임

Photon PUN2 기반 스냅샷 보간 물리 퍼즐 동기화

Unity C# Photon PUN2

05



UE4 숨바꼭질 PvP

Unreal Engine 4 기반 1:4 비대칭 멀티플레이 PvP

Unreal Engine 4 C++ Replication



[Gameplay Video](#)

Project 1: Dream Catcher 모바일 비주얼 노벨

XML 스크립트 파싱 엔진과 모바일 최적화 시스템을 구축, 완성된 게임을 플레이 스토어에 출시

프로젝트 개요

장르:	모바일 비주얼 노벨 (싱글 플레이)
플랫폼:	Android (Google Play Store 출시)
기간/인원:	2021.03 ~ 2022.03 (약 1년) / 총 7인 (클라이언트 2인)
사용 기술:	Unity, C#

핵심 구현 목표 :

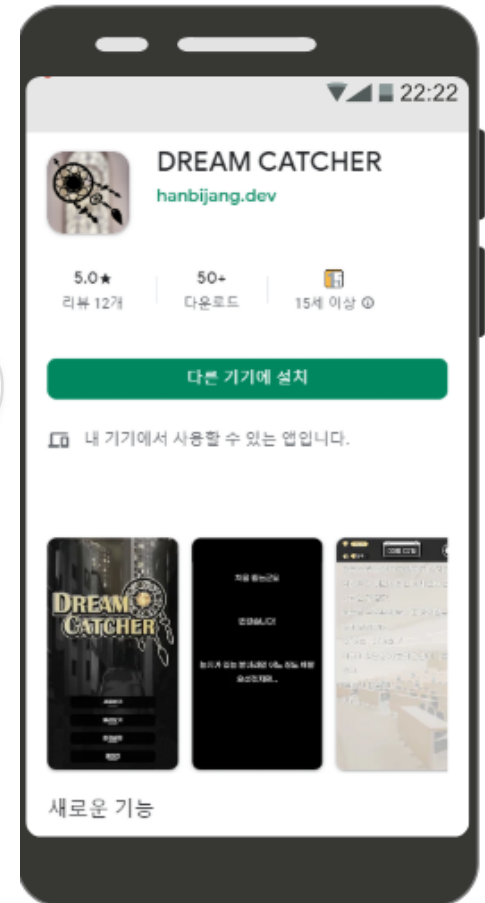
- 자체 커스텀 XML 파싱 엔진 아키텍처 설계 및 구현
- 데이터 주도형 멀티 엔딩 분기 시스템 구축
- JSON 직렬화 활용 다중 슬롯 세이브 및 Observer 패턴 기반 업적 시스템 구현
- 모바일 환경에 최적화된 터치 이벤트 처리 및 UI/UX 시스템 연출



[Source Code \(일부\)](#)



[Gameplay Video](#)

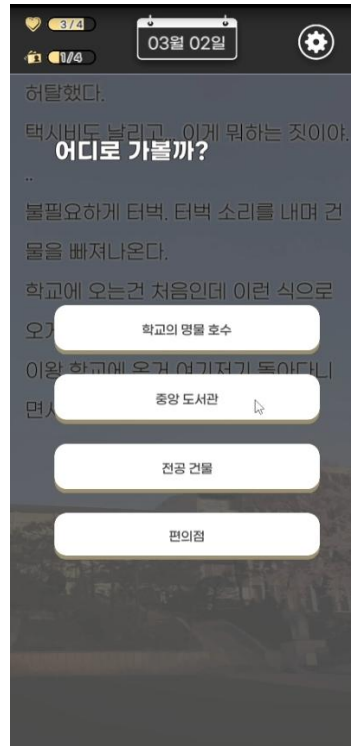


Google Play Store 출시 이력

링크를 통해 소스 코드 및 시연 영상을 확인하실 수 있습니다

Play Screenshots

커스텀 XML 엔진 기반의 유연한 시나리오 분기와 모바일 환경에 최적화된 핵심 시스템 구현



XML 엔진 기반의 대화 및 타이핑 연출

직접 구현한 XML 파싱 엔진이 스크립트의 ID, 태그, 조건을 단계별로 처리하여 텍스트를 출력하는 화면
코루틴을 활용한 글자별 타이핑 효과와 큐(Queue) 자료구조를 이용한 대사 관리 및 스킵 로직

태그 시스템을 활용한 멀티 엔딩 선택지

레이어의 선택에 따라 런타임에 태그 값을 변경(SetTg)하고 조건(CondCheck)을 평가하여 시나리오 분기를 제어하는 장면

XML, JSON 기반 데이터 관리

데이터의 성격에 맞춘 업적, 갤러리 해금 등의 데이터 저장 관리 시스템으로 안정성을 확보

Core Tech 1: 커스텀 XML 파싱 시스템

게임 로직과 콘텐츠를 완전히 분리하여 스크립트 파일 교체만으로 유연한 업데이트가 가능한 시스템 구축

커스텀 파싱 시스템 설계

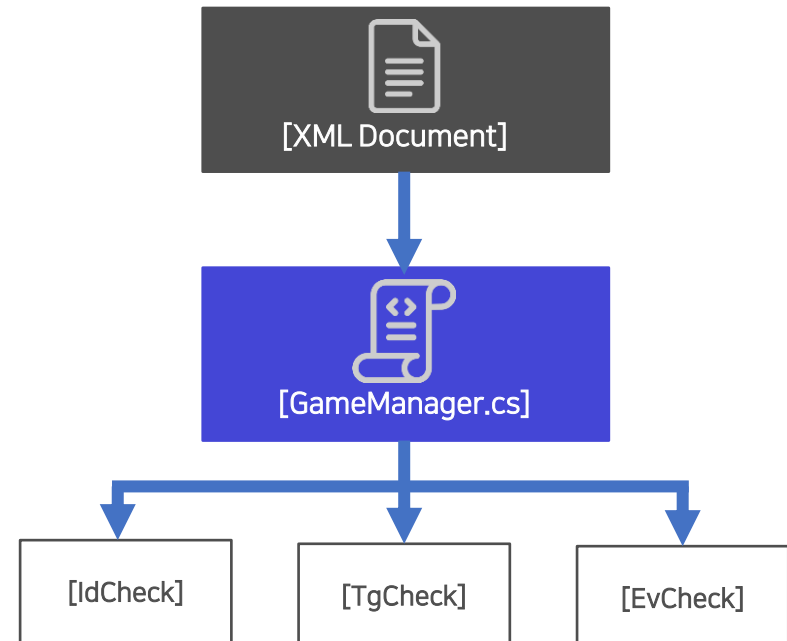
- 외부 에셋에 의존하지 않고 GameManager.cs 내에 커스텀 XML 파서 직접 구현
- 텍스트 출력, 분기 조건 처리, 이벤트 실행 등 모든 인게임 요소를 XML 태그로 구조화

단계별 데이터 파싱 파이프라인

- IdCheck: 캐릭터 및 대화 ID 매칭을 통한 화자 식별
- TgCheck & RandCheck: 런타임 태그 변경 및 난수 기반 이벤트 발생 처리
- EvCheck: 사운드, 이펙트, 씬 전환 등의 시스템 이벤트 호출

기술적 기대 효과

- 기획자가 코드 수정 없이 XML 스크립트만으로 콘텐츠 전체를 제어 가능
- 대용량 텍스트 처리의 작업 병목을 줄이고 콘텐츠 확장성을 극대화



Core Tech 2: 데이터 주도형 멀티 엔딩 시스템

런타임 시 태그 조합과 조건 평가를 통해 복잡한 분기 로직을 구현

동적 태그(Tag) 할당 및 조건 평가 로직

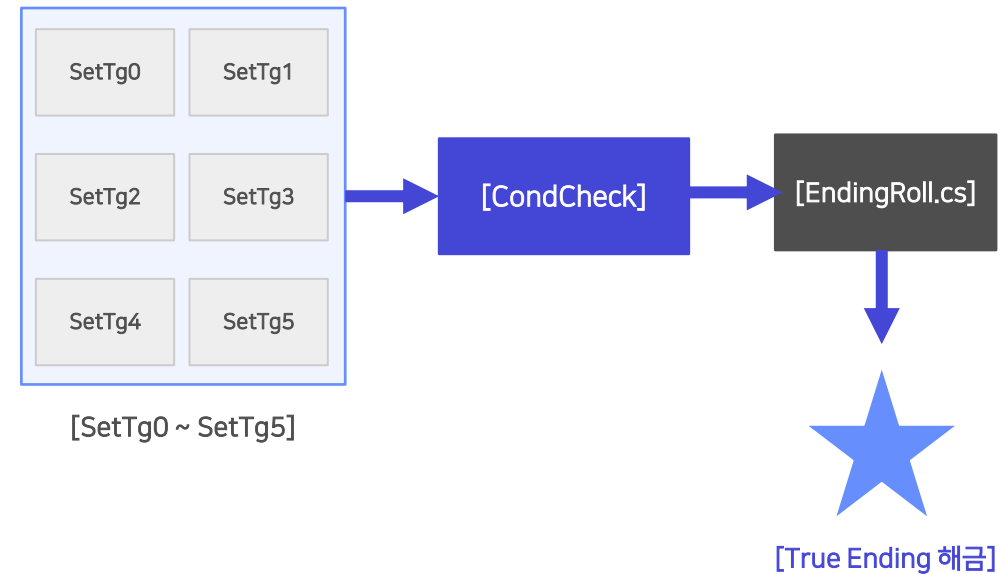
- SetTg0 ~ SetTg5 메서드를 통해 유저의 선택지에 따라 런타임 태그 값을 실시간 변경
- CondCheck 로직으로 누적된 태그 조합을 평가하여 엔딩 분기 조건 산출

자동화된 엔딩 처리 시스템

- EndingRoll.cs가 엔딩 태그(ending)를 감지하여 독립적인 스텝프를 연출 실행
- 엔딩 도달 시 일러스트 갤러리의 특정 이미지 해금 데이터를 연동하여 자동 갱신 처리

확장성 및 유연성 확보

- 단순 true/false 플래그가 아닌 구조화된 태그 조합으로 다층적 스토리텔링 로직 제어
- 신규 엔딩 추가 시 기존 코드 수정 없이 XML 분기 조건만 수정하여 대응 가능



Core Tech 3: 패턴 및 직렬화 기반 데이터 관리

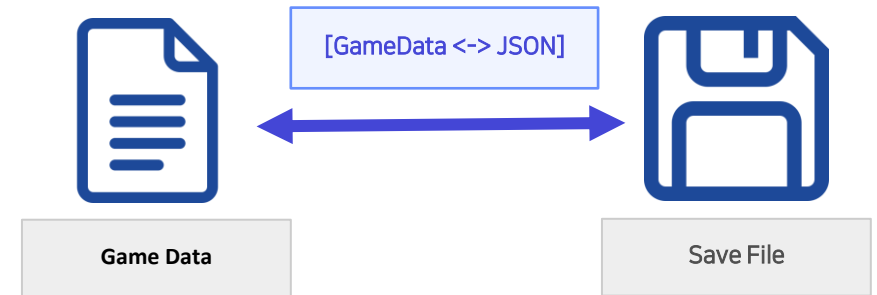
다중 세이브 슬롯 시스템과 업적 데이터, 데이터 관리의 안정성 확보

세이브 / 로드 시스템 (JSON 직렬화)

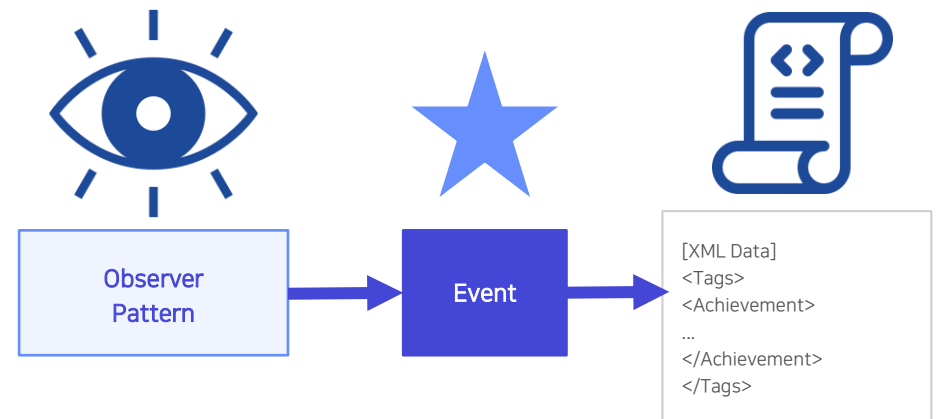
- DataManager.cs에서 ObjectToJson을 활용해 GameData 객체를 **JSON** 구조로 직렬화
- SlotManager와 연동하여 다중 슬롯 저장 기능을 지원하고, 빠른 저장(QuikSave)을 분리 설계

업적 달성 시스템 (Observer 패턴 + XML)

- AchievementManager.cs에 OnNotify 메서드를 구현하여 게임 내 이벤트를 수신하는 **Observer** 패턴 적용
- 조건(CompareCond) 충족 시 달성 여부를 판단하여 결합도를 낮춘 이벤트 처리
- AchievementXmlManager.cs를 통해 업적 데이터를 **XML**로 상태 저장
- **Queue** 기반 업적 순차 처리 - 동시 다중 업적 달성 시 Queue로 순차 연출 보장



세이브 로드 시스템



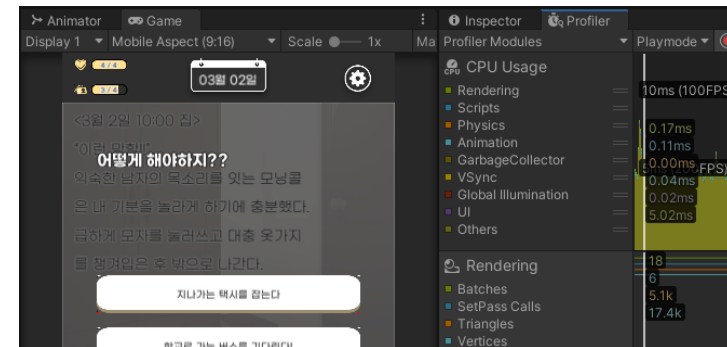
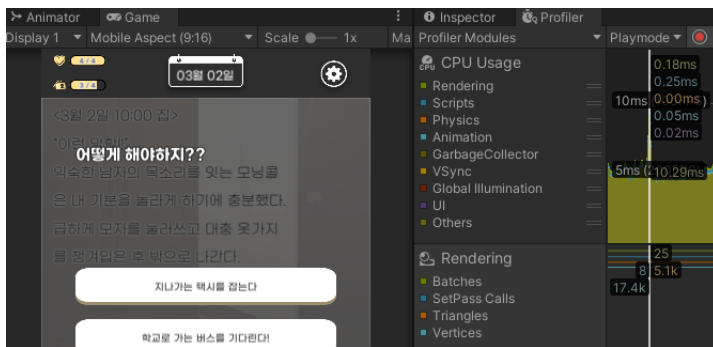
업적 달성 시스템

Core Tech 4: 모바일 최적화

다양한 모바일 사양 환경을 위한 모바일 최적화 적용

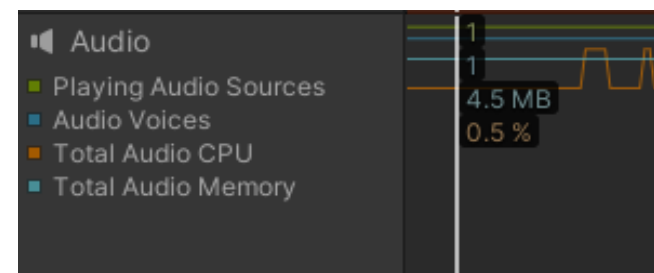
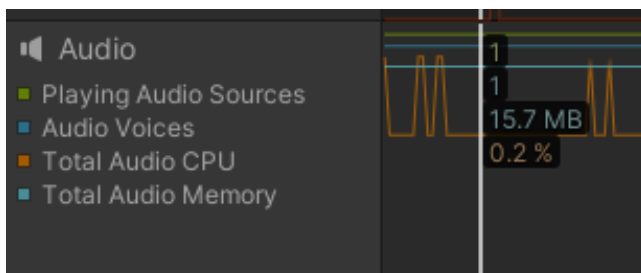
Sprite Atlas를 활용한 드로우콜 최적화

- 메인 게임 씬에서 같은 상황에 Batches : 25 -> 18로 **28% 감소** / SetPass Calls : 8->6 감소
- Sprite Atlas 패킹 리스트 : 메인 게임 씬의 UI 이미지



게임 BGM - 메모리 사용량 최적화

- 약 20곡 이상의 BGM들의 Load Type : Decompress on Load -> Streaming 변경 오디오 메모리 사용량을 15.7MB -> 4.5MB로 **71% 감소**
- CPU 사용률은 0.2% -> 0.5%로 소폭 증가했으나 무시 가능한 수준으로, 모바일 환경에 더 적합하다고 판단



Troubleshooting: GC Alloc 메모리 최적화

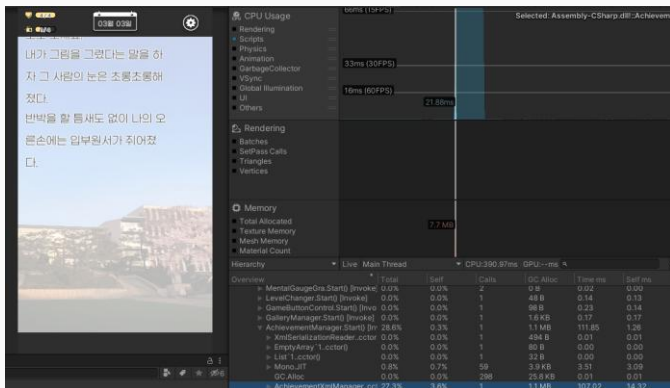
게임 대화 진행 시, 일정 수준으로 프레임이 떨어지는 현상 발생과 이에 대한 해결

Before

대화 텍스트가 진행되는 동안 주기적으로 프레임 드랍 / 끊김 현상 발생

- 게임 '이어하기' 기능에서 멈춤, 프레임 드랍 발생
- [1] 대량의 11726번 GC Alloc Calls 호출 횟수 확인
- 이후 Profiler로 분석하여 Xpath, XML 기능이 문제 중 하나라는 것을 확인
- [2] 고정적으로 업적 XML 처리 생성자에서 처리 시간 지연 (약 100ms) 확인

	Total	Self	Calls	GC Alloc	Time ms
SoundHandle.Instance.CreateChannel	0.0%	0.0%	1	0 B	0.00
Task`1..cctor()	0.0%	0.0%	1	312 B	0.13
CodePageDataItem..cctor()	0.0%	0.0%	1	34 B	0.00
List`1..cctor()	0.0%	0.0%	1	80 B	0.00
Mono.JIT	5.2%	4.9%	434	21.4 KB	18.87
Loading.ReadObject	11.2%	0.0%	53	0 B	40.93
Instantiate	2.5%	0.0%	40	0.9 MB	9.35
GC Alloc	0.1%	0.1%	11726	0.7 MB	0.49
FindObjectsOfType	0.0%	0.0%	1	0 B	0.00
StatusManager.UpdateMoGaugeCoroutine() [Coroutine: Sys	0.0%	0.0%	1	0 B	0.01
StatusManager.UpdateMoGaugeCoroutine() [Coroutine: Mo	0.0%	0.0%	1	0 B	0.06
StatusManager.UpdateMoGaugeColorCoroutine() [Coroutine:	0.0%	0.0%	1	0 B	0.01

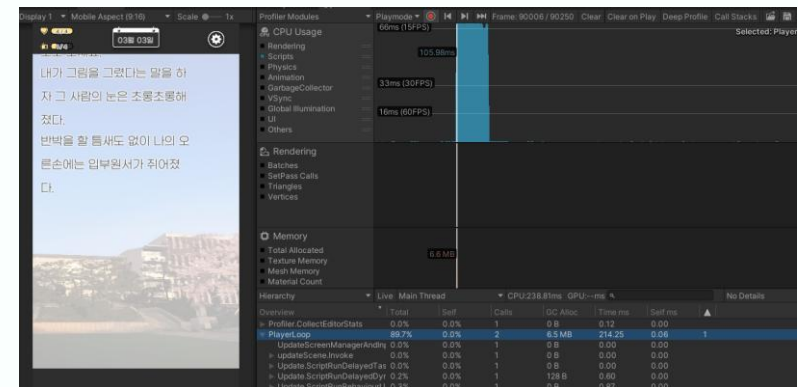


After

XML파서 기능을 XPath방식에서 인덱서 방식으로 변경

- [1] XPath 방식의 SelectSingleNode(~) 에서 인덱서 방식으로 변경 (node["ID"])
- 같은 조건에서, GC Alloc Calls 11726번 -> 2758번으로 감소
- GC Alloc이 0.7MB에서 221.3KB로 유의미한 감소
- [2] 업적 XML 처리자 생성을 static화 하여 오픈 로딩 화면으로 미리 생성 시점 옮김 -> 약 100ms만큼 처리 속도 빨라짐

	Total	Self	Calls	GC Alloc	Time ms
ContentSizeFilter.OnRectTransformDimensionsChange()	0.0%	0.0%	1	0 B	0.02
Transform.SetParent	1.2%	0.1%	40	6.0 KB	4.55
XPathNodeList..cctor()	0.0%	0.0%	1	32 B	0.00
XmlReader..cctor()	0.0%	0.0%	1	0 B	0.00
XmlDocument..cctor()	0.0%	0.0%	1	288 B	0.07
XmlConvert..cctor()	0.0%	0.0%	1	78 B	0.00
XPathNavigator..cctor()	0.0%	0.0%	1	236 B	0.02
XPathParser..cctor()	0.0%	0.0%	1	5.6 KB	0.33
SoundManager.GetHandle	0.3%	0.0%	1	0 B	1.19
SoundHandle.Instance.CreateChannel	0.0%	0.0%	1	0 B	0.00
Task`1..cctor()	0.0%	0.0%	1	312 B	0.13
CodePageDataItem..cctor()	0.0%	0.0%	1	34 B	0.00
List`1..cctor()	0.0%	0.0%	1	80 B	0.00
Mono.JIT	4.8%	4.5%	435	21.4 KB	18.16
Loading.ReadObject	12.3%	0.0%	53	0 B	46.85
Instantiate	2.2%	0.0%	40	0.9 MB	8.62
GC Alloc	0.0%	0.0%	2758	221.3 KB	0.11
FindObjectsOfType	0.0%	0.0%	1	0 B	0.00
StatusManager.UpdateMoGaugeCoroutine() [Coroutine: Sys	0.0%	0.0%	1	0 B	0.02



Project 2: 2D 소울라이크 전투 데모

패링, 스테미나, 보스 2페이지 등 소울라이크 장르의 핵심 전투 시스템을 구현

프로젝트 개요

장르:	2D 소울라이크 액션 RPG
플랫폼:	PC (Windows)
기간/인원:	2025.08 ~ 2026.01 (약 6개월) / 인원 1인
사용 기술:	Unity, C#, FSM

핵심 구현 목표 :

- FSM 기반 아키텍처 설계
- 반응형 보스 AI와 2페이지 각성
- 패링, 스테미나, 콤보 시스템
- 데이터와 로직의 분리



Source Code



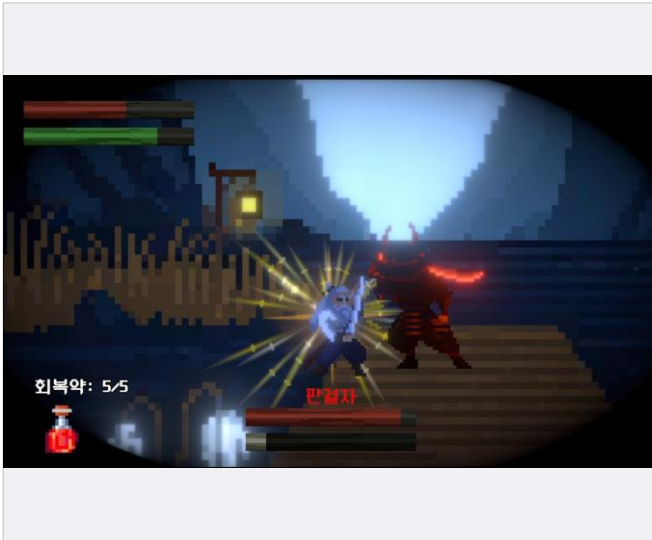
Gameplay Video

링크를 통해 소스 코드 및 시연 영상을 확인하실 수 있습니다



Play Screenshots

보스와 패링, 스테미나, 보스 2페이지가 발생하는 전투



패링 연출

정확한 타이밍을 요구하는 패링 성공 시의 충격파 이펙트
및 카메라 셰이크 연출



보스 2페이지

체력 50% 이하에서 발생하는 보스의 2페이지 각성 광폭
화 패턴



콤보 액션

플레이어의 다단 콤보 공격과 스테미나 소비가 유기적으
로 연동됨

Core Tech 1: FSM 기반 아키텍처 설계

PlayerStateMachine을 설계하고 모든 등장 인물 합산 약 30개의 독립 상태 클래스로 행동을 분리하여 코드의 결합도를 감소

객체 지향적 상태 분리

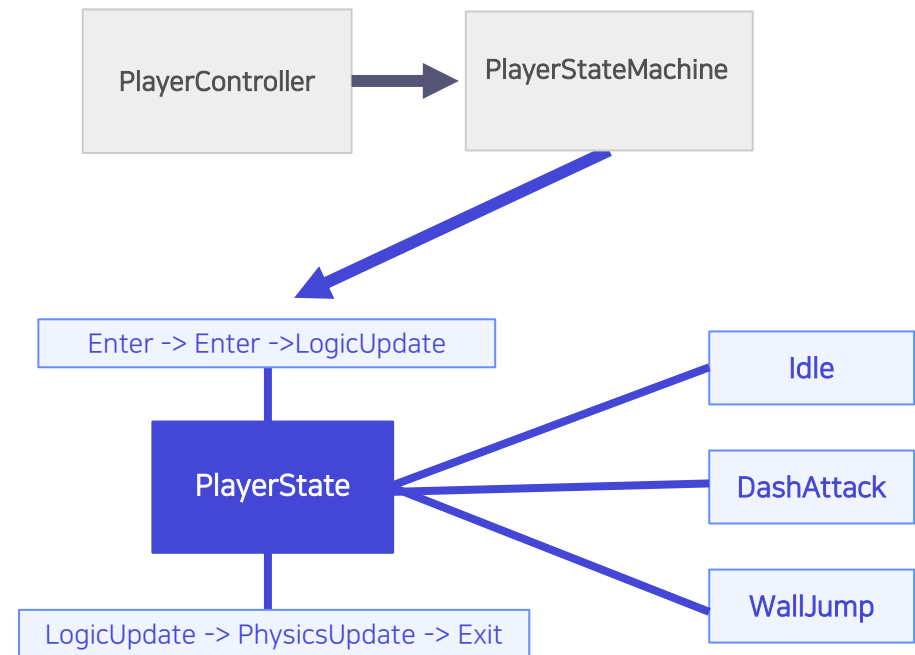
- PlayerState 베이스 클래스 기반으로 Idle, DashAttack, WallJump 등 행동 세분화
- 각 상태가 독립적인 클래스로 분리되어 단일 책임 원칙(SRP) 준수

명확한 생명주기 관리

- Enter → LogicUpdate → PhysicsUpdate → Exit 사이클로 물리 연산과 로직 분리
- 프레임 단위 물리 처리와 게임 로직 처리를 분리하여 예측 가능한 동작 보장

확장성 확보

- 새로운 액션 추가 시 기존 코드를 수정하지 않고 새 클래스 상속만으로 구현 가능
- OCP(개방-폐쇄 원칙) 적용으로 유지보수 비용 최소화



[PlayerStateMachine 구조와 상태 분리]

Core Tech 2: 반응형 보스 AI와 2페이지 각성

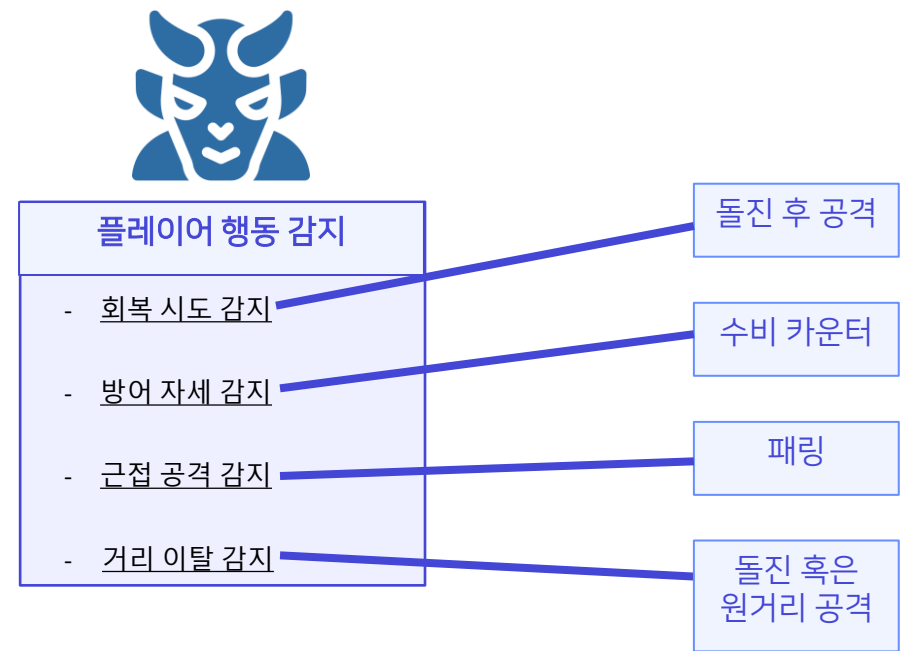
플레이어의 상태를 실시간으로 감지하여 반응하는 보스 몬스터의 AI를 구축

반응형 보스 : 실시간 플레이어 상태 감지 반응

- `IsPlayerHealing()` 감지 시 -> 돌진 공격 즉시 실행
- `IsPlayerDefending()` 감지 시 -> 딜레이 카운터 공격으로 전환
- `EnemyParryState` : 플레이어의 공격을 감지하여 보스가 먼저 패링을 시도
- 딜레이 시간: `minActionDelay` ~ `maxActionDelay` 랜덤 간격으로 기계적 패턴 탈피

2페이지 각성 시스템 (EnrageState)

- 체력 50% 이하 도달 시 각성 애니메이션 1회 보장 재생 후 패턴 전환
- 이동속도 및 공격력 수치 강화, 공격 패턴 다양화 적용
- `EnrageState`를 FSM 상태로 분리하여 전환 로직의 명확성 확보



[반응형 보스]

Core Tech 3: 패링, 스테미나, 콤보 시스템

장르 특유의 정밀한 패링 타이밍, 스테미나 소비, 콤보 연계 구현

정밀한 패링 판정

- 방어 입력 초반 parryWindow(0.15초) 내 적 공격 감지 시 패링 성립 및 특수 이펙트 출력
- 카메라 셰이크, 히트 스톱 연출로 타격감 극대화

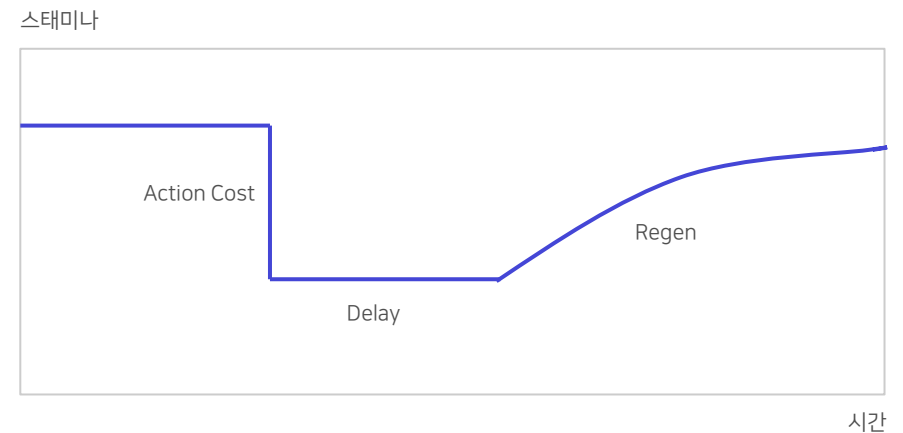
0.15초 (Parry Window) - 정확한 타이밍 요구



[패링 판정]

유기적인 스테미나 관리

- 달리기, 점프, 대시, 공격 등 모든 행동에 스테미나 부여 및 딜레이 후 자동 회복
- 스테미나 부족 시 특정 행동 잠금으로 자원 관리의 전략적 요소 추가



[스테미나 관리]

데이터 기반 콤보 공격

- AttackData 배열을 통한 다단 콤보 구성 및 comboResetTime 내 연속 입력 판정 처리
- ScriptableObject로 공격 데이터를 외부화하여 기획자 친화적 밸런싱 환경 구성



[공격 콤보 구성]

Core Tech 4: 데이터와 로직의 분리

ScriptableObject로 데이터를 분리하여, 게임 밸런싱이 가능하도록 설계

캐릭터 스탯 수치 관리 (PlayerStatsSO)

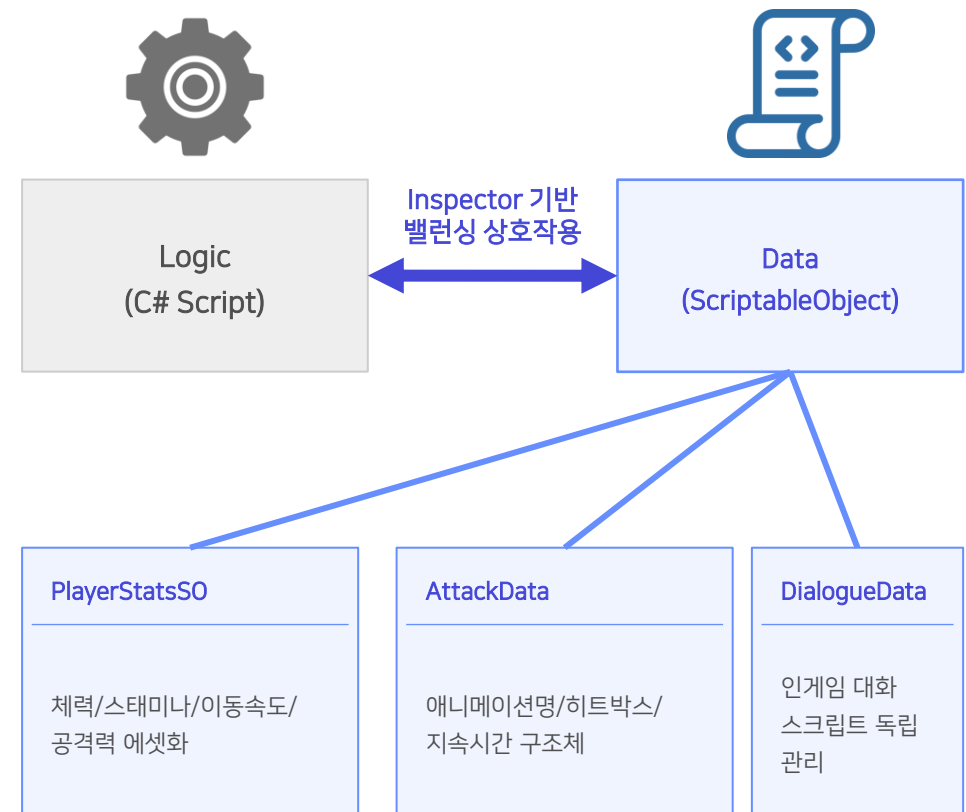
- 체력, 스테미나, 이동속도, 공격력 등 모든 핵심 수치를 에셋화하여 에디터 Inspector에서 조정
- 런타임 코드 변경 없이 밸런싱 가능 -> 빠른 작업 가능 설계

공격 데이터 캡슐화 (AttackData)

- 애니메이션 이름, 공격 지속 시간, 히트박스 오프셋을 구조체로 관리하여 액션 동작 확장 용이
- 공격 타입별 독립 에셋으로 관리하여 재사용성 확보

대화 시스템 데이터화 (DialogueData)

- 말풍선 UI와 연동되는 인게임 대화 스크립트를 독립적으로 관리하여 텍스트 수정 편의성 증가



[데이터와 로직의 분리]

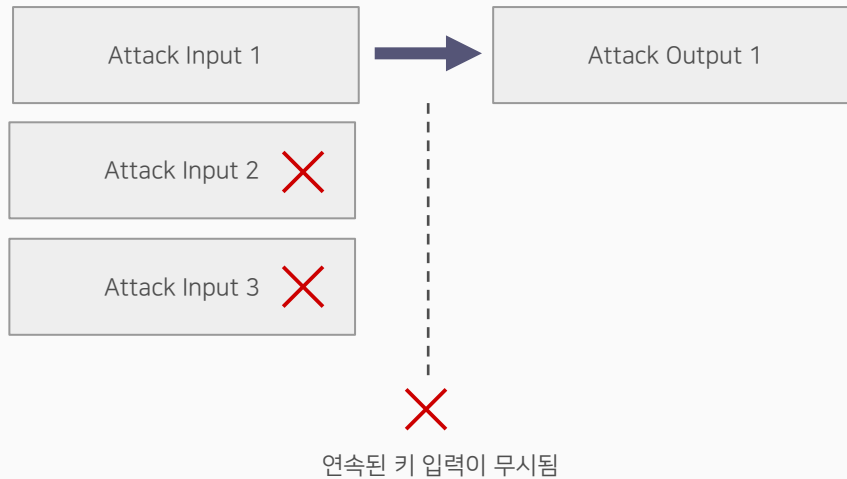
Troubleshooting: 게임 플레이 경험 개선

연타가 주된 전투 시스템에서 입력이 무시되는 상황이 자주 발생, 입력 버퍼로 해결

Before

연속된 키 입력이 필요한 전투에서 다량의 입력이 무시됨

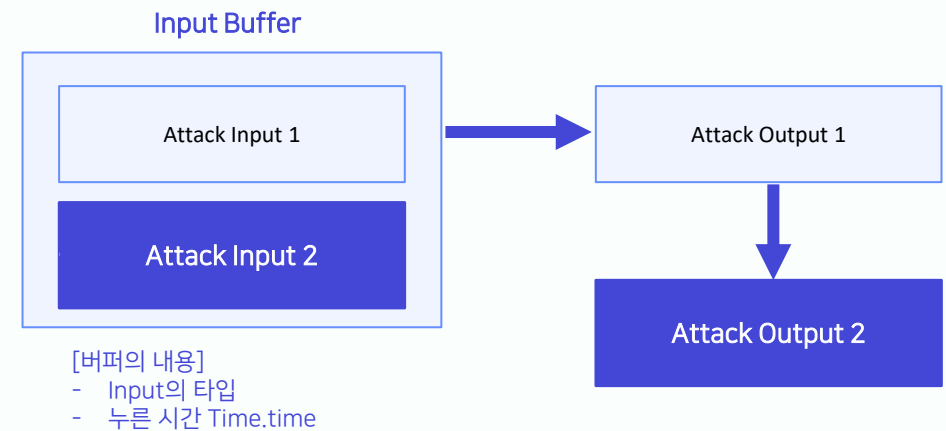
연속된 키 입력이 필요한 전투 상황에서, 공격 버튼을 여러 번 눌러야 공격이 1회 발생하는 등, 키 입력이 무시되는 상황 발생 -> 게임의 조작감이 떨어짐



After

입력 버퍼 방식으로 플레이 경험 향상

플레이어의 Input을 타입 별로 버퍼에 적재 -> 처리가 가능할 때 Consume함수로 처리
-> 플레이어가 누른대로 output이 발생 - 조작감, 플레이 경험 향상



Project 3: 리듬 레이싱 멀티플레이 게임

BPM 동기화 노트 판정과 3개의 레인 이동을 결합한 2인 멀티플레이 게임 프로젝트

프로젝트 개요

장르:	리듬 레이싱 멀티플레이어
플랫폼:	PC (Windows)
기간/인원:	2023.01 ~ 2023.02 / 팀 4인, 이후 개인 리팩토링
사용 기술:	Unity, C#, C++, TCP Socket

핵심 구현 목표:

- C++ 기반 다중 접속 서버 구축
- 커스텀 패킷 헤더 및 데이터 직렬화 - 클라이언트 통신 시스템 연동
- Command 패턴 기반 클라이언트 통신 제어
- 서버 시간 기반 노트 동기화와 판정 시스템
- 동적 시각 연출
- 커스텀 인게임 노트 에디터

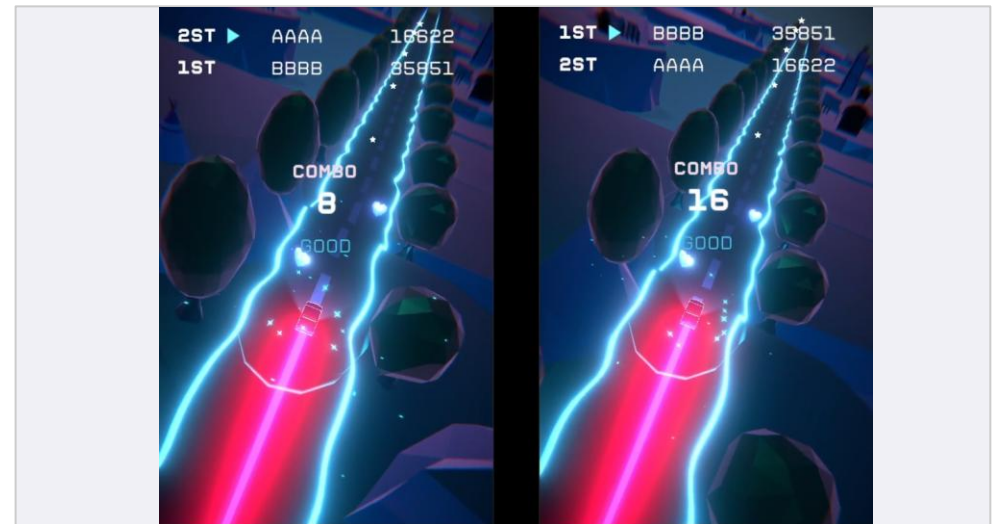


Source Code



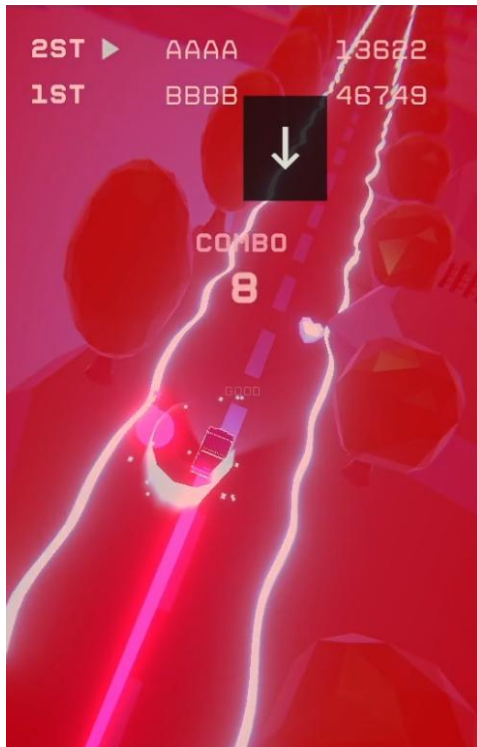
Gameplay Video

링크를 통해 소스 코드 및 시연 영상을 확인하실 수 있습니다



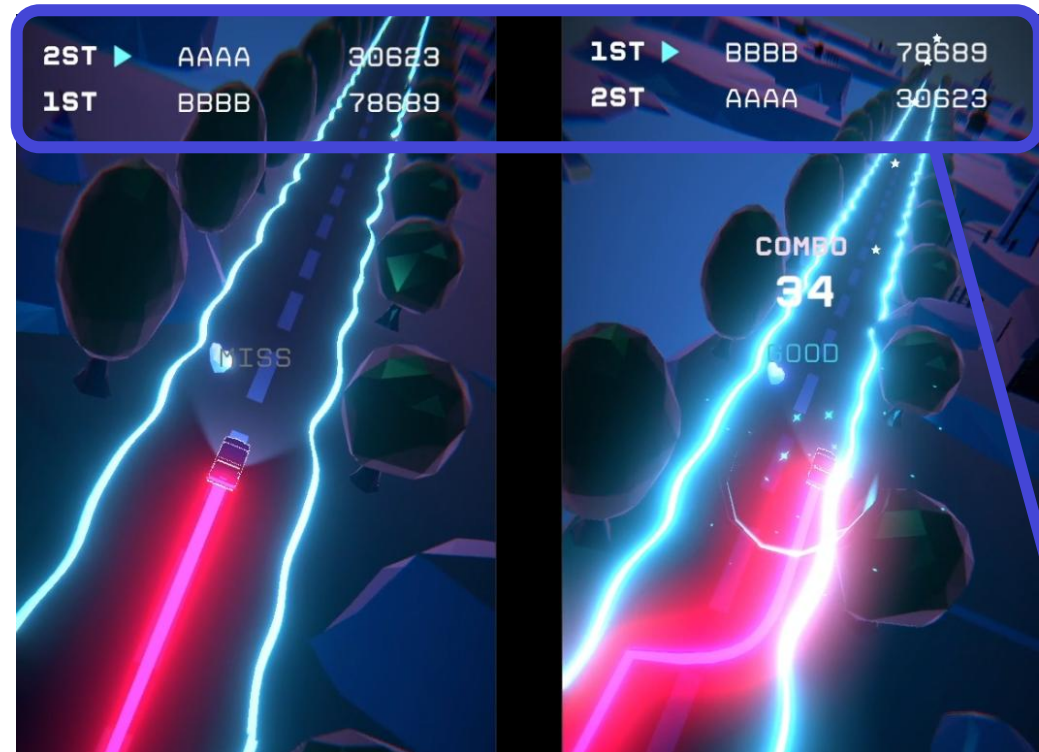
Play Screenshots

TCP 서버 동기화 + 실시간 오디오 분석 기반 시각 연출



서버 동기화 기반의 실시간 상호작용

생성된 노트들 과 실시간으로 생성된 상대의 방해 공격으로 인해 타격을 받은 플레이어



콤보 반응형 환경 연출 및 포스트 프로세싱 효과

[좌: 적용 전, 우: 적용 후]

플레이어의 콤보에 맞춰 씬의 조명(Ambient Light)과 스카이박스 노출도가 변하고, 높은 콤보 수치로 인해 화면에 Bloom과 모션 블러 효과가 강하게 적용

플레이어 점수 동기화

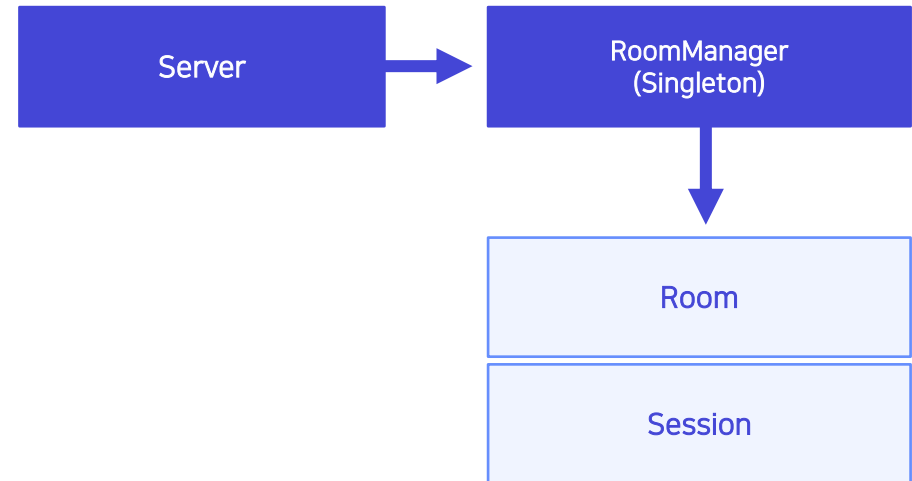
현재 경쟁 중인 다른 플레이어들의 닉네임과 실시간 점수 변화가 표시되는 인게임 UI

Core Tech 1: C++ 기반 다중 접속 서버 구축

Server, Session, Room의 3계층 구조로 설계하여 네트워크 패킷 처리의 효율성과 확장성을 확보

매치 메이킹 3계층 구조 설계

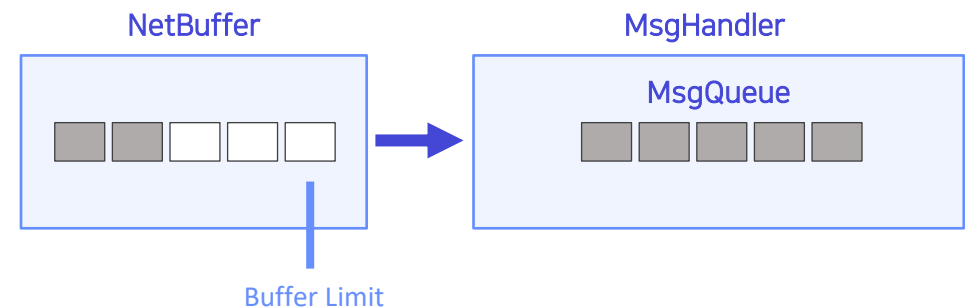
- **Server**: 접속 대기(Accept) 및 전역 Session 할당 관리
- **RoomManager(Singleton)**: 멀티플레이 룸 생성 및 실시간 매치메이킹 조율
- **Room**: 노드 스폰·판정 처리 및 인게임 상태 관리
- **Session**: 개별 클라이언트 소켓 연결 유지 및 비동기 송수신 처리



효율적인 메시지 핸들링 방법

- **MsgHandler** 내부 글로벌 메시지 큐 (std::mutex 기반 Push/Pop 보호)를 통한 스레드 안전성 확보
- **NetBuffer**: 버퍼 오버플로우 방지 및 안전한 바이트 조작 지원
- **Boost.Asio** 라이브러리 기반 비동기 네트워크 I/O 안정 처리

[매치 메이킹 구조 설계]



[메시지 핸들링 매커니즘]

Core Tech 2: Command 패턴 기반 클라이언트 통신 제어

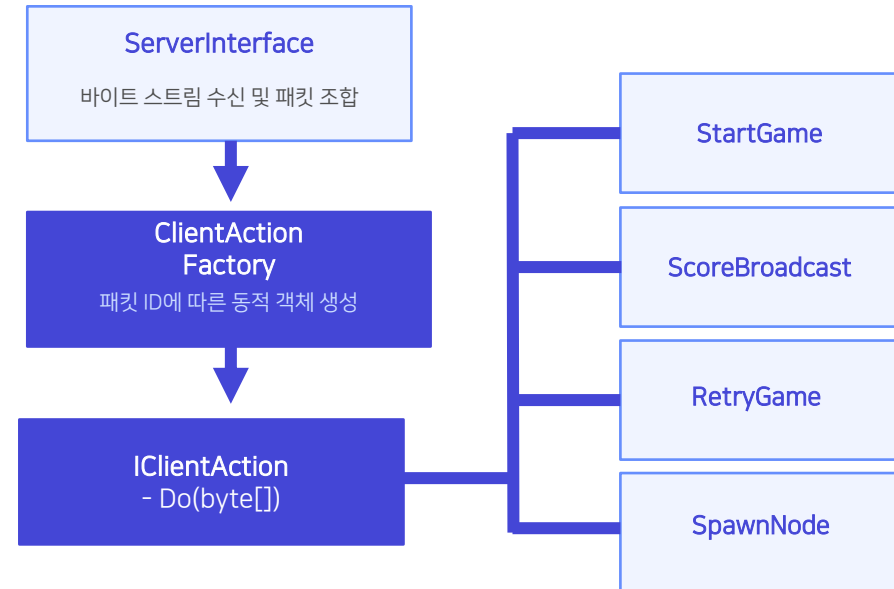
IClientAction 인터페이스를 도입하여 서버에서 수신한 다양한 타입의 패킷을 확장성 있게 처리

의존성 분리 및 확장성

- ServerInterface에서 수신된 패킷을 ClientActionFactory로 전달
- 패킷별 독립 클래스 분할로 OCP(개방-폐쇄 원칙) 준수
- 새로운 패킷 추가 시 팩토리 분기만 추가하여 유지보수 확장성

전역 상태 관리 (ClientState)

- ClientState Enum 기반으로 완전한 게임 루프 상태 통제
- 각 상태별 명확한 UI 및 통신 로직 격리
- Lobby → Matching → InGame 상태 전이로 실행 전환 안정성 확보



[클라이언트에서 수신하는 패킷 데이터 구조]



[게임 루프 구조]

Core Tech 3: 게임 성능 최적화 - 오브젝트 풀링

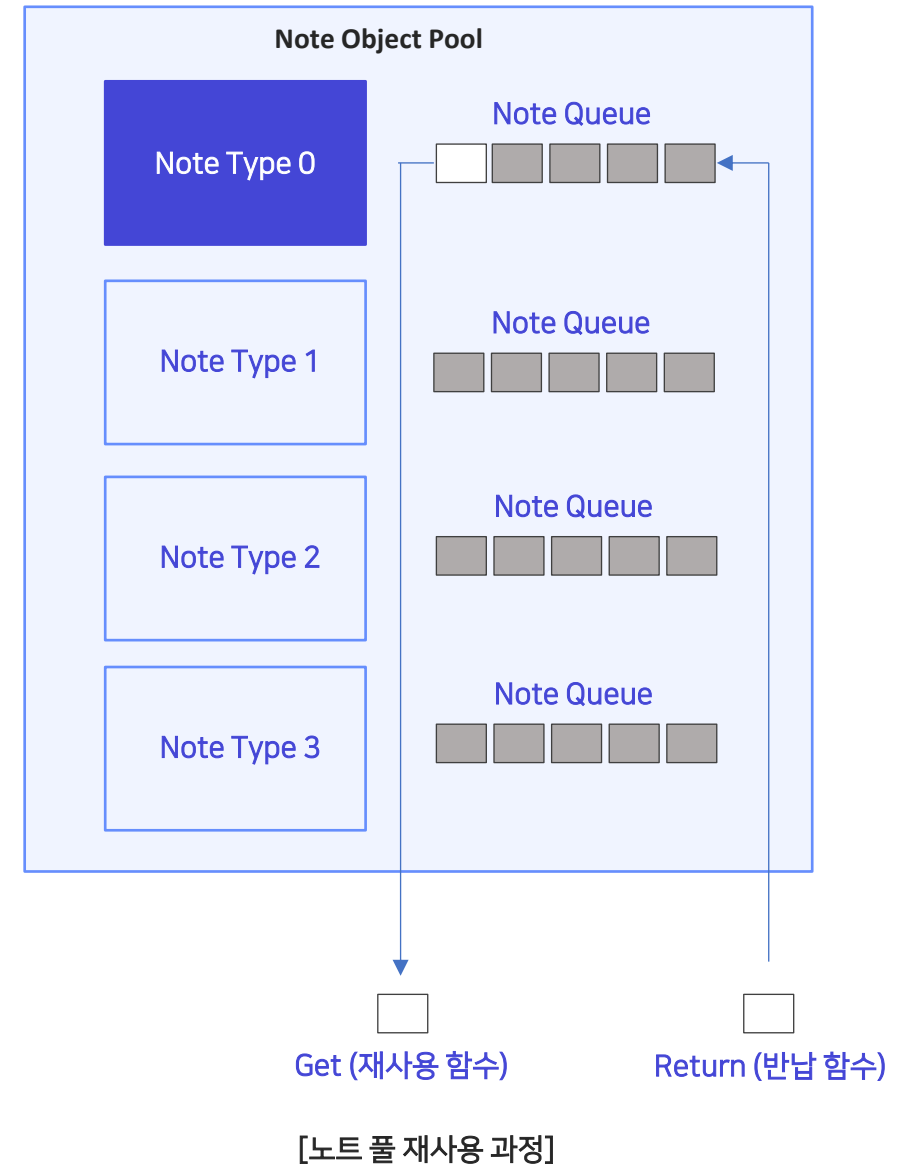
리듬 게임 특성 상, 반복 생성, 삭제가 필요한 구조를 최적화하기 위해서 오브젝트 풀링 기법 사용

최적화 이전 상황

- 노트가 등장할 때마다 Instantiate -> 메모리 할당
- 노트가 사라질 때마다 Destroy -> GC(가비지 컬렉션) 발생
- 리듬 게임 특성상 짧은 시간에 반복 -> 프레임 드랍 유발

오브젝트 풀링을 리듬 노트에 적용

- 게임 시작 시 노트를 미리 생성해 풀(Queue)에 대기
- 필요할 때 큐에서 꺼내어 사용 (SetActive true), 끝나면 큐에 반납(SetActive false)
- 예외 상황을 제외하고, Instantiate/Destroy 호출 없음
 - 예외 상황 : 오브젝트 풀이 없거나, 풀이 모두 사용중일 때



Core Tech 4: 동적 시각 연출

오디오 주파수 실시간 분석으로 조명, 스카이 박스, 이펙트 동적 제어

오디오 주파수 분석 (BeatReactor)

- AudioSource.GetSpectrumData() 호출로 실시간 주파수 배열 획득
- 실시간 베이스 주파수 및 파형 데이터 추출

환경 동적 제어

- 추출된 진폭에 맞춰 Directional Light 및 Ambient Light 동적 제어
- Skybox Exposure 값을 비트에 맞춰 조정하여 배경 반응형 연출

포스트 프로세싱 및 이펙트

- 콤보 비례 Bloom 강도 및 Motion Blur 상승 처리
- 플레이어 이동 시 반투명 메시 복제본을 활용한 GhostTrail 잔상 생성



[게임 시각 연출 구현]

Core Tech 5: 커스텀 인게임 노트 에디터

Unity 에디터 내에서 노트를 손쉽게 편집할 수 있는 기능

에디터 개요

- 별도의 외부 툴 없이 Unity 씬 내에서 동작하는 노트 에디터 구현
- 음악을 재생하며 A / S / D 키 입력으로 레인별 타이밍을 실시간 녹음
- 녹음 완료 후 서버가 읽는 XML 포맷으로 자동 저장

핵심 로직 : 스폰 시점 역산

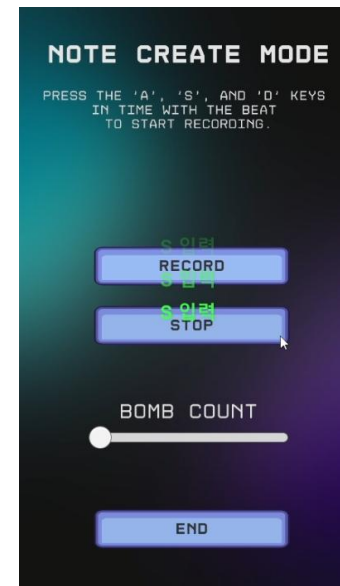
- 녹음 시 기록되는 시간 = 플레이어가 노트를 치는 히트 타이밍
- 서버는 히트 타이밍이 아닌 스폰 시점을 기준으로 노트를 생성
- 스폰 시점 = 히트 타이밍 - (노트 이동 거리 ÷ 노트 이동 속도)
플레이어와 노트 사이의 거리와 이동 속도값을 우선 참조하여 타이밍 계산

Fail 노트 자동 배치 (Bomb Count 조절)

- 녹음한 노트가 없는 빈 타이밍 구간 탐색 -> 랜덤으로 Fail 노트 배치

히트 타이밍 입력 녹음
(A/S/D 녹음)

Note Time 기록 XML 문서



이동 시간 역산

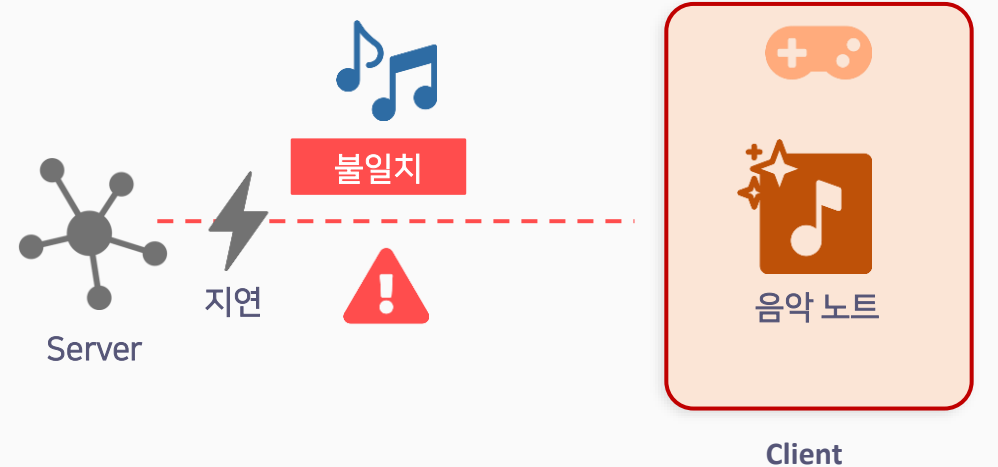
스폰 시점 계산

```
<?xml version="1.0" encoding="utf-8"?>
<AllNode>
  <Node>
    <Time>3.901</Time>
    <Type>1</Type>
    <Pos>0</Pos>
  </Node>
  <Node>
    <Time>4.455</Time>
    <Type>1</Type>
    <Pos>2</Pos>
  </Node>
  <Node>
    <Time>5.010</Time>
    <Type>1</Type>
    <Pos>0</Pos>
  </Node>
  <Node>
    <Time>5.543</Time>
    <Type>2</Type>
    <Pos>2</Pos>
  </Node>
  <Node>
    <Time>6.119</Time>
    <Type>1</Type>
    <Pos>0</Pos>
  </Node>
  <Node>
    <Time>6.695</Time>
    <Type>2</Type>
    <Pos>2</Pos>
  </Node>
</AllNode>
```

[에디터 녹음 화면과 생성된 XML 결과 일부]

Troubleshooting: 리듬 노트 싱크 불일치 보정

서버의 프레임 타이머 보정과 클라이언트의 시간 역산 로직으로 리듬 동기화 문제 해결

Before	After
음악과 리듬 노트의 싱크 차이 발생	서버의 FramTimer 도입, 클라이언트의 startTimeUtcMs
- 문제 1) 클라이언트 기기에 프레임 드랍 발생 시 음악 리듬과 노트의 도달 타이밍 어긋남 - 문제 2) 네트워크 지연(Latency)으로 인해 미세한 지연이 겹쳐, 유저마다 노트 스폰 타이밍이 다르게 체감	- 문제 1 해결) 서버: FrameTimer 도입, expectedTime (프레임이 시작되어야 할 예상 시간)과 measureTime(현재 시간) 비교로 지연 누적 시 프레임 스킵 -> 일정한 주기로 SpawnNode 패킷 생성, 리듬 일관성 확보 - 문제 2 해결) 클라이언트: StartGameData 패킷과 함께 오는 startTimeUtcMs (동기화 시작 시간)으로 동일한 리듬 타임 라인을 공유
	 기기 성능 및 네트워크 핑과 무관하게 노트가 음악 비트에 정확히 맞춰 도달

Project 4: 물리 퍼즐 멀티플레이 게임

Photon PUN2를 활용하여 2D 물리 연산이 동반되는 2인 퍼즐 게임

프로젝트 개요

장르: 2D 물리 퍼즐 멀티플레이 게임 (수박 게임 모티브)
플랫폼: PC (Windows)
기간/인원: 2024.04 ~ 2024.05, 2026.04 / 인원 1인
사용 기술: Unity, C#, Photon PUN2

핵심 구현 목표 :

- 인게임 경험 향상 및 씬 전환 아키텍처
- 스냅샷 보관 기반 물리 동기화
- 네트워크 지연 대응
- 동작 별 통신 프로토콜 구분



Source Code



Gameplay Video

링크를 통해 소스 코드 및 시연 영상을 확인하실 수 있습니다



Play Screenshots

Photon PUN2 기반 실시간 물리 동기화, 2인 대전 퍼즐 게임



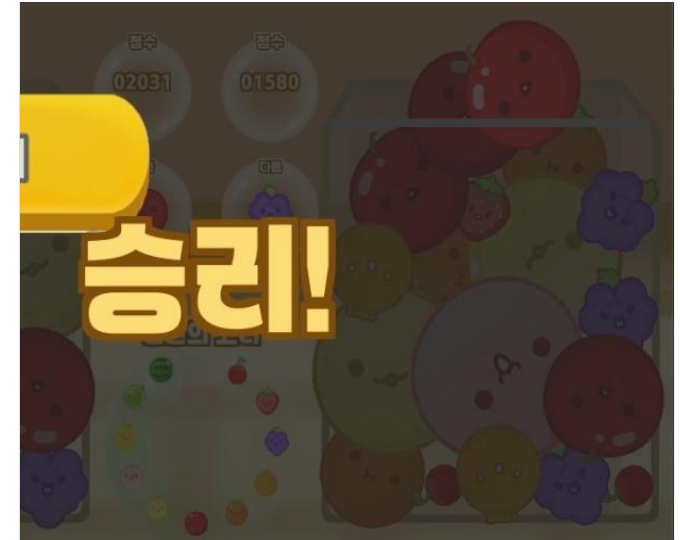
대기 로비에서의 실시간 매칭

방 생성, 이름 검색 참가, 빠른 매칭. 2인 입장을 감지하여 매치 시작을 동기화하는 화면



물리 동기화 기반의 과일 합체 플레이

로컬 물리 시뮬레이션과 0.1초 간격의 위치 스냅샷 전송(Unreliable)을 결합하여, 상대방 화면에서도 끊김 없이 부드럽게 과일이 쌓이고 합체함



최종 승패 판정

양쪽 플레이어의 게임오버 시점을 동기화하여 최종 점수를 비교하고 승패를 결정

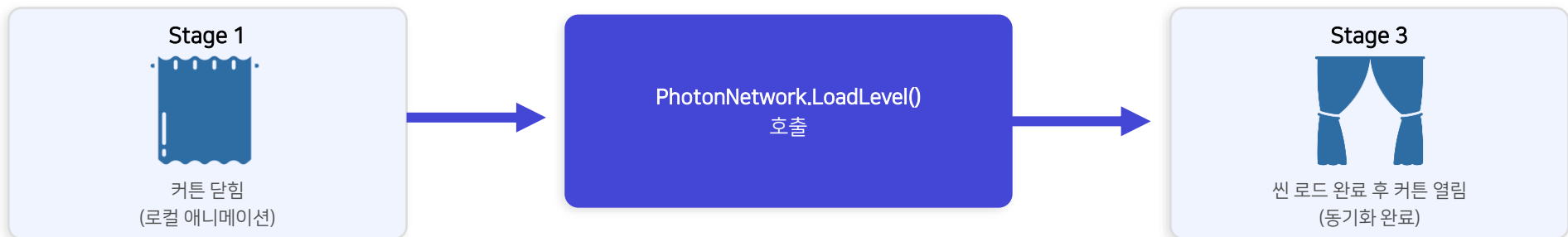
Core Tech 1:인게임 경험 향상 및 씬 전환 아키텍처

커튼 애니메이션을 활용해 네트워크 로딩 지연을 시각적으로 자연스럽게 함

멀티플레이 씬 분리: MultiGameManager가 멀티 전용 이벤트를 독립적으로 처리

동기화된 씬 전환: 방장의 LoadLevel 호출 한 번으로 모든 클라이언트가 동시에 씬 이동 (AutomaticallySyncScene = true)

자연스러운 로딩 연출: LoadingSceneController의 커튼 닫힘/열림 애니메이션으로 씬 로드 및 네트워크 동기화 시간을 자연스럽게 커버



StartAutoTransitionToMulti() 실행 흐름

Core Tech 2: 스냅샷 보간 기반 물리 동기화

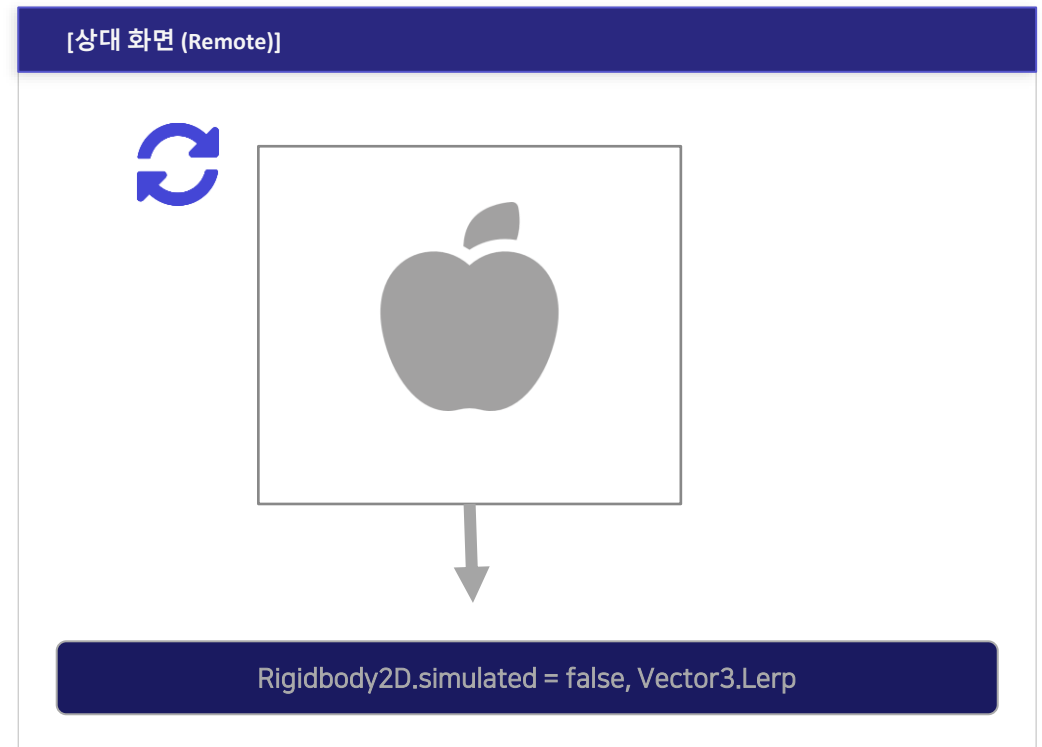
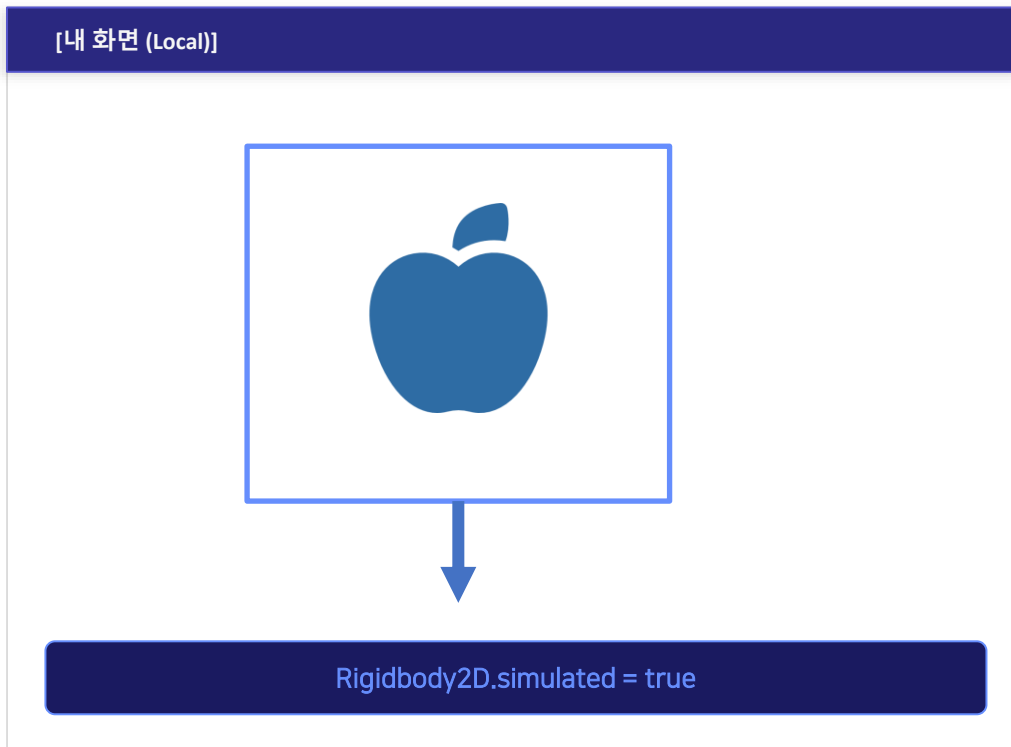
로컬에서만 물리 엔진을 구동하고 상대방 화면은 연산을 배제한 위치 스냅샷으로 렌더링

비대칭 시뮬레이션: 각 클라이언트는 자신의 과일만 Rigidbody2D로 실제 연산 수행

상대방 객체 제어: 상대방 과일의 simulated 및 Collider2D를 완전히 비활성화

Lerp 보간 렌더링: 네트워크로 수신받은 위치와 회전 스냅샷 데이터(networkTargetLocalPos)를 $\text{Time.deltaTime} \times 20f$ 속도로 부드럽게 추적

로컬의 과일 객체와 리모트 과일 객체의 물리 엔진 처리 방식 비교



Core Tech 3: 네트워크 지연 대응

방장이 매치 시작 이벤트를 2가지 방식으로 병렬 전송, 네트워크 지연 및 늦은 입장 상황 대처

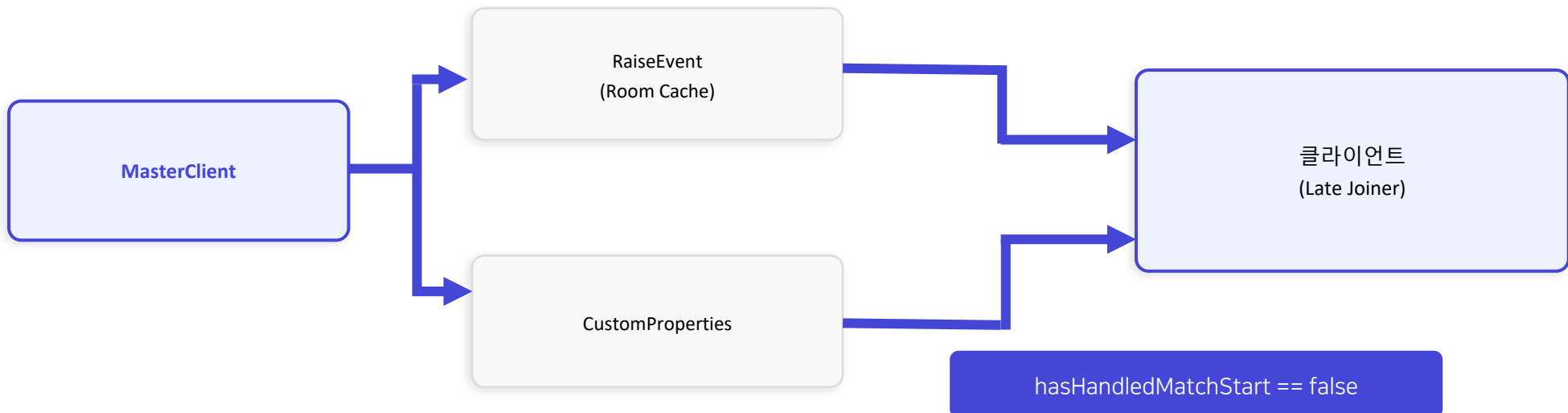
조건 감지: MasterClient가 룸 인원 2명 도달 감지 후 0.5초 대기

방법 A (이벤트 캐싱): RaiseEvent 호출 시 EventCaching.AddToRoomCache 옵션을 적용해 늦은 입장자도 수신 가능하도록 룸 캐시에 적재

방법 B (프로퍼티 갱신): Room CustomProperties["MatchStarted"] = true 설정으로 이벤트 수신 전 입장하거나 패킷이 유실된 경우 OnRoomPropertiesUpdate로 복구

중복 방지: 2가지 방법 모두 콜백 시, hasHandledMatchStart 플래그를 검사하여 로직의 중복 실행 차단

[아키텍처 플로우] 매치 시작 신호의 병렬 처리 및 복구



Core Tech 4: 동작 별 통신 프로토콜 구분

데이터 특성에 맞춰 전송 신뢰성(Reliability)을 분리하고 고유 카운터로 네트워크 양단의 객체를 일치

지연 최소화 전략: 0.1초마다 연속 전송되는 EVT_FRUIT_STATE는 패킷이 유실되어도 다음 패킷으로 보정되므로 Unreliable 방식으로 전송

상태 보장 통신: 과일 생성, 머지, 게임 오버 등 단발성 중요 이벤트는 Reliable 전송

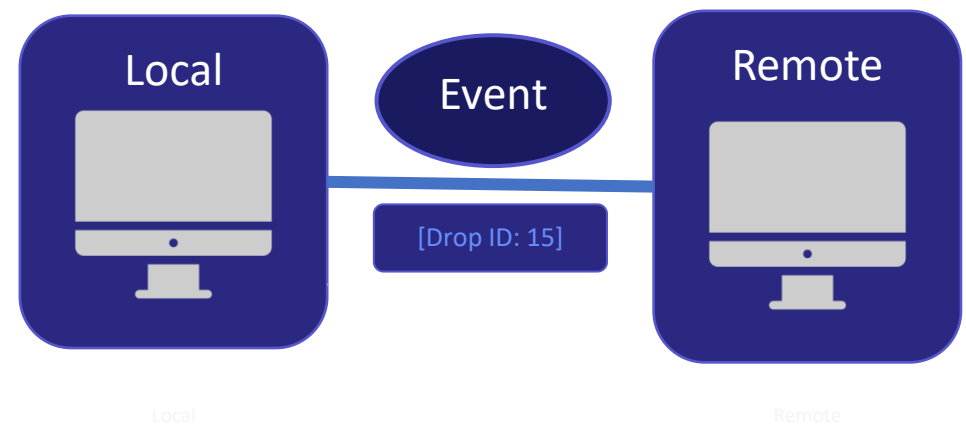
고유 식별자 부여: FruitManager가 생성 시점마다 ID 발급

동기화 매핑: 상대방 측 게임 화면 동기화를 위해, Dictionary<int, Fruit>로 Drop ID와 과일 객체를 매핑하여 타겟 갱신

네트워크 통신 오버헤드 최적화를 위한 이벤트 타입 분류

이벤트 코드	전송 방식
EVT_FRUIT_CREATE (과일 생성) EVT_FRUIT_MERGE (과일 합체) EVT_FRUIT_DROP (과일 낙하) EVT_GAME_OVER (게임 오버)	Reliable
EVT_FRUIT_STATE	Unreliable

[통신 프로토콜 구분]



[이벤트 ID 발급]

Troubleshooting: 물리 연산 비용 및 지연 보정

상대방 물리 연산 제거 + 즉시 스프라이트 갱신으로 불일치 해결



Project 5: UE4 숨바꼭질 PvP

언리얼 엔진 4의 Replication 시스템을 활용해 1:4 비대칭 멀티플레이 환경을 구축한 PvP 프로젝트

프로젝트 개요

장르:	멀티플레이 PvP 숨바꼭질 게임
플랫폼:	PC (Windows)
기간/인원:	2022.04 ~ 2022.08 / 인원 1인
사용 기술:	Unreal Engine 4, C++

핵심 구현 목표 :

- Replication & RPC 아키텍처 설계
- 생존자 - 동적 소품 변신 시스템
- 공포 연출 및 시스템 설계



Source Code



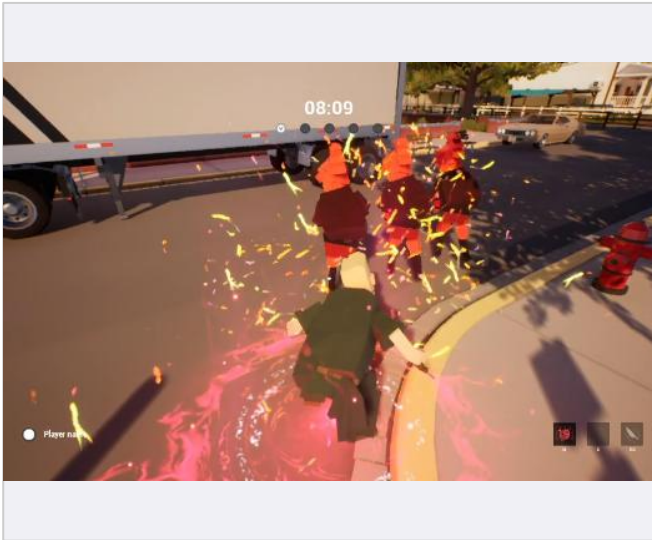
Gameplay Video

링크를 통해 소스 코드 및 시연 영상을 확인하실 수 있습니다



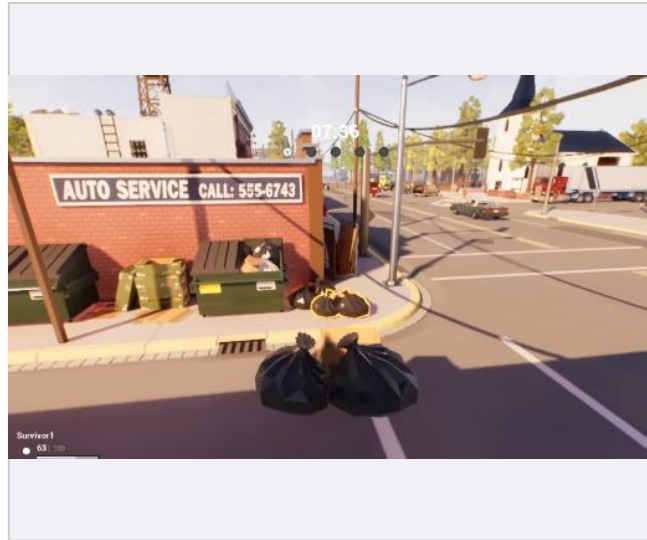
Play Screenshots

킬러의 추적과 생존자의 전략적인 은신이 요구되는 서바이벌 게임



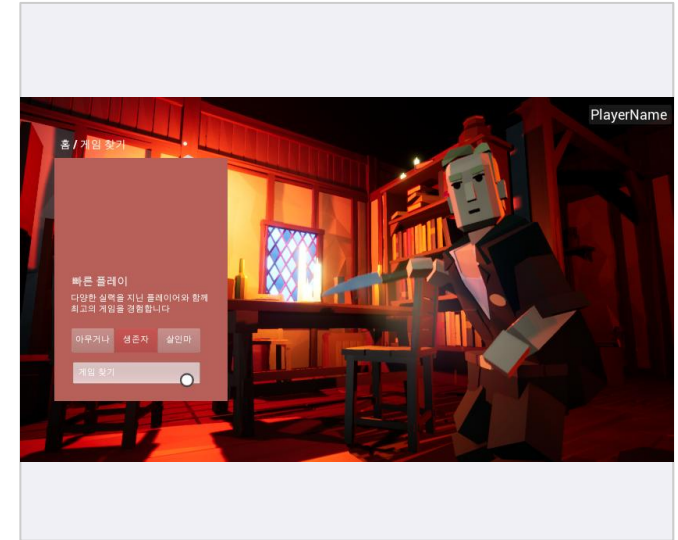
Killer's Perspective:

원거리 공격(Q), 특수기(E), 원격 조작 투사체(RC) 스킬 활용



Survivor's Perspective:

맵 내의 소품 변신을 통한 은신



Matching Lobby:

실시간 매칭 시스템 및 5명 정원 충족 시 자동 게임 레벨 전환

Core Tech 1: Replication & RPC 아키텍처 설계

Server, NetMulticast, Client RPC의 역할을 분리하여 최적화된 클라이언트-서버 동기화 구조를 설계

RPC 권한 통제

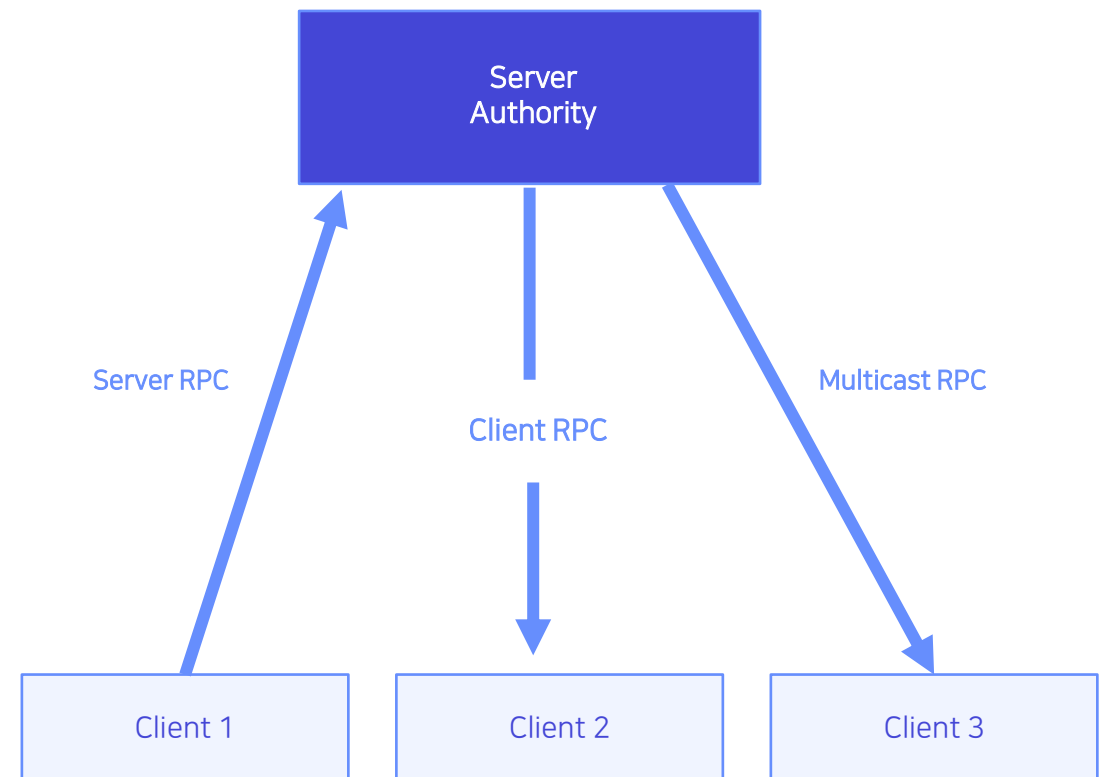
- UFUNCTION(Server): 데미지 연산, 쿨타임 계산 등 중요 로직의 서버 중앙 처리
- UFUNCTION(NetMulticast): 이펙트 스폰, 애니메이션 등 모든 클라이언트의 시각적 피드백 동기화
- UFUNCTION(Client): 개별 플레이어 UI 업데이트 전송으로 대역폭 최적화

네트워크 프로퍼티 동기화

- UPROPERTY(ReplicatedUsing = OnRep_*): 상태 변이 시 클라이언트 자동 갱신
- GetLifetimeReplicatedProps() 오버라이드로 복제 속성 정밀 관리

데이터 구조화

- FSurvivorInfo, FKillerInfo 구조체를 Replicated 선언해 클라이언트 HUD와 즉시 연동



[게임 동기화 구조]

Core Tech 2: 동적 소품 변신 시스템

메시 교체 + 물리 제어로 소품 변신 구현

생존자 플레이어 형상 변환 메커니즘

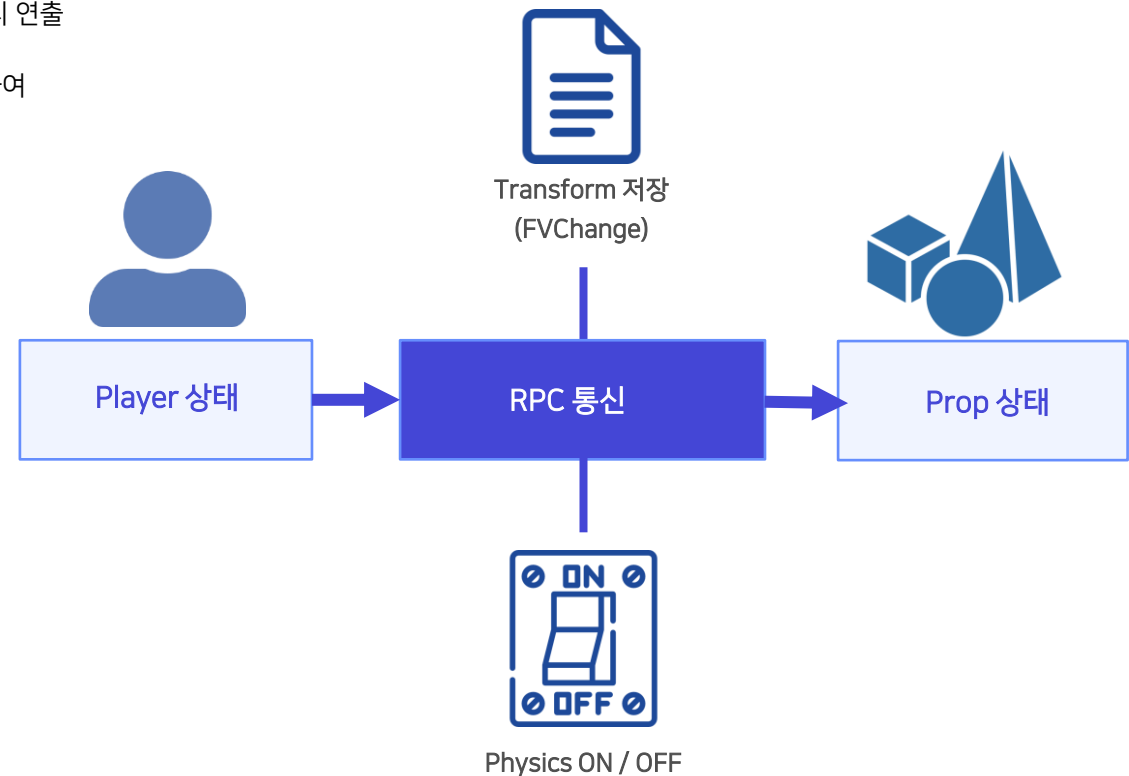
- ChangeToObject 호출 시 캐릭터 메시 비활성화 및 스택틱 메시(소품) 활성화
- SetSimulatePhysics()로 물리 시뮬레이션을 동적 활성화/비활성화하여 소품 물리 연출
- 변신 상태 진입 시 거리 판정 기준을 캐릭터에서 소품 오브젝트 위치로 자동 전환하여 변신 중 킬러 감지 정확도 보장

위치 복원 안정성 확보

- 변신 직전의 위치와 회전값을 FVChange, FRChange 변수에 저장
- 변신 해제 시 오차 없이 원위치로 복원하여 지형 뚫림 방지

쿨타임 제어

- bChangeEnable 플래그 기반의 쿨타임 시스템으로 스킬 남용 제어



[생존자 플레이어 형상 변환 메커니즘]

Core Tech 3: 공포 연출 및 시스템 설계

거리 기반 포스트 프로세싱 연출 + DataTable 스탯 관리

동적 공포 연출 시스템 (ShowVinetting)

- 킬러가 500 유닛 이내 접근 시 카메라 포스트 프로세스 동적 활성화
- 거리(Distance) 함수에 비례하여 Vignette 강도 실시간 변화
- 거리 기반 심장 소리 및 추적 음악 볼륨 제어

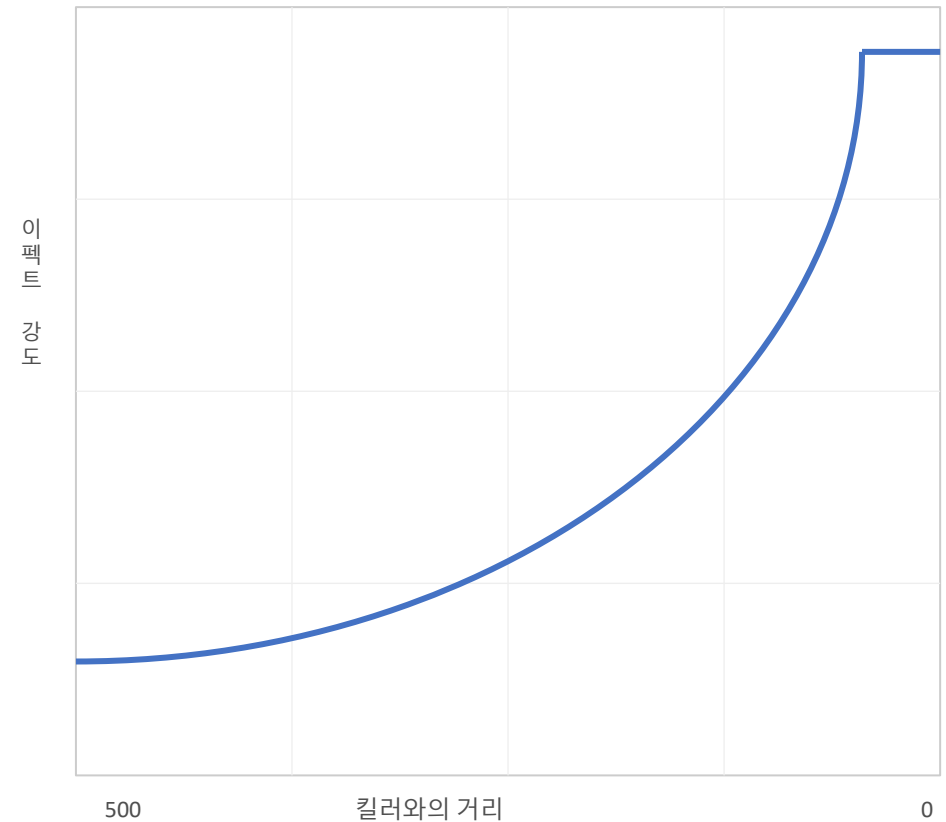
데이터 주도 스탯 관리

- DataTable 기반 설계(DT_Survivor, DT_Killer)로 하드코딩 배제 및 기획적 밸런싱 용이성 확보

안정적인 매칭 및 레벨 전환

- PostLogin 이벤트에서 MatchingPCArr 카운팅으로 5명 정원 감지
- 타이머 기반 OpenLevel("GameMap") 호출로 무결성 높은 자동 레벨 전환

Activation Curve Graph



[UI, 오디오, 포스트 프로세싱의 로직]

Troubleshooting: 네트워크 권한 및 이동 동기화

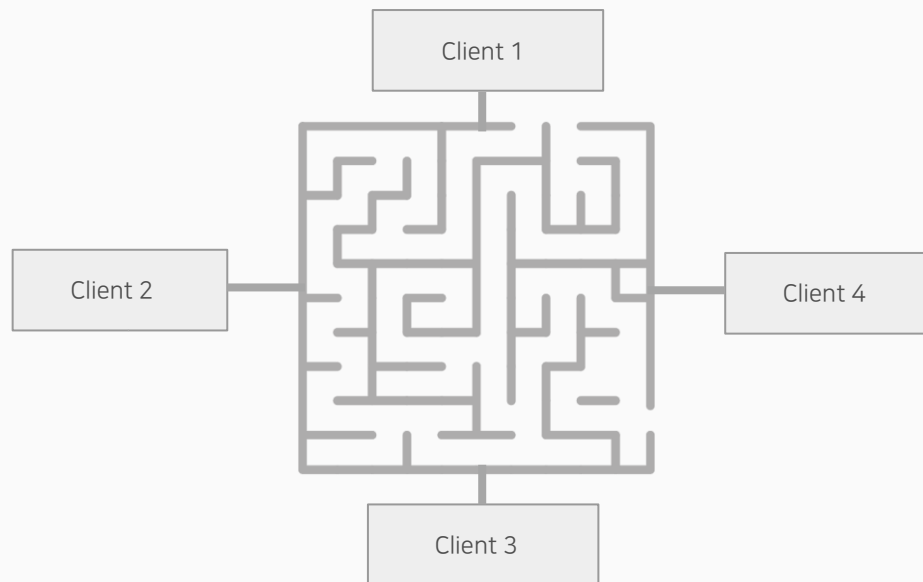
클라이언트 간 이동 동기화 불일치 현상을 Server RPC 중심의 권한 제어 패턴으로 재설계하여 해결

Before

클라이언트 간 상태 변경 불일치

초기 구현 시 **클라이언트** 단에서 독립적으로 움직임과 상태를 계산
-> 네트워크 지연으로 인해 모든 플레이어의 상태 변경이 클라이언트 간에 심각하게 어긋남

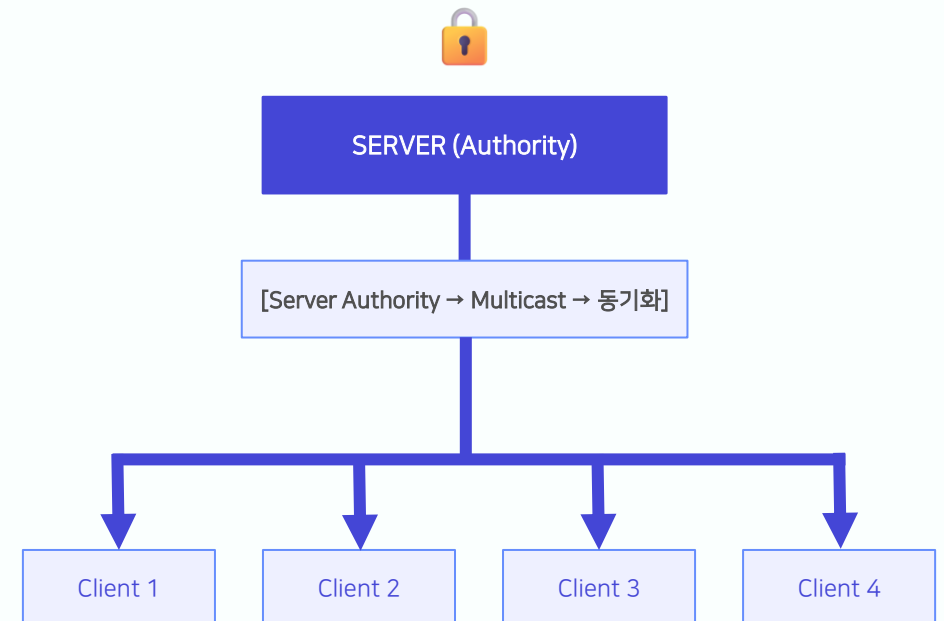
[Client Authority → Desync 발생]



After

해결 과정 (Refactoring) & 결과 도출

- 클라이언트 간 모든 움직임 입력을 직접 처리하지 않고 **Server RPC**로 전송
- **서버가 권한(Authority)**을 가지고 위치 연산 후 **Multicast**
- OnRep_State 콜백이 일관되게 호출되어 어긋남 감소
- **권한 일원화**를 통한 네트워크 동기화 무결성 확보





읽어주셔서 감사합니다.

PORTFOLIO

Name: 장한비 (Hanbi Jang)

Contact : 010-5227-5728 | hanbijang.dev@gmail.com

GitHub : github.com/HanbiJang

Core Strength : 설계 패턴의 실제 적용 및 C++ TCP, UE4 Replication, PUN2 멀티플레이 환경 구축

